OWNER-AUTHORIZED / PRE-RELEASE / FROZEN SOURCE

Groupmine 2.3.0

Security and Operational-Correctness Review

One shared distributor / two independent custody pools

QUALIFIED REVIEW CONCLUSION

Two-pool lifecycle verified

Production consideration remains gated by identity verification, known compiler exposure and actual launch-input evidence.

FROZEN COMMIT

`02ef1f7da17683b482b97b28464346bc2cb9db61`

REVIEWER

ChatGPT, GPT-6 Astra Pro

IMMEDIATE BASELINE

`bdf906f801f59acefc49094361a530b823a73ab8`

REVIEW DATE

31 October 2026 UTC

SCOPE

Production, setup/tooling, canary, dependencies and policy

Frozen implementation unchanged. No production credentials accessed. No public-chain transaction signed or broadcast. This review is not a deployment authorization or SALUS closure.

Contents

Complete review, including evidence limits and release gates. Section entries and PDF bookmarks are clickable.

1. Executive conclusion . . . 3

2. Scope, provenance and evidence discipline . . . 4

3. Initialization: executable order and irreversible boundaries . . . 7

4. Independent economic reasoning and observed results . . . 9

5. Distribution and failure isolation . . . 12

6. Findings and confirmed setup boundaries . . . 14

7. Operational correctness, gas and tool behavior . . . 17

8. Inherited risks, compiler and canary separation . . . 19

9. Fresh execution ledger and limits . . . 21

10. Coverage matrix . . . 24

11. Prioritized disposition and remaining launch inputs . . . 25

12. Evidence navigation, reproduction and source references . . . 27

13. Direct answers to the owner's six questions . . . 29

Edition note: This is the complete English review of frozen Groupmine 2.3.0 dated 31 October 2026 UTC. It distinguishes fresh execution, source reasoning, author-recorded evidence and unexecuted checks. The six direct owner questions are answered in section 13.

Evidence companion: GROUPMINE_02ef1f7_REVIEW_EVIDENCE.zip. Detailed results, proof sources, failed-attempt records and the SHA-256 manifest remain in that separate archive.

Report identity and scope

Frozen target: `02ef1f7da17683b482b97b28464346bc2cb9db61`

Immediate baseline: `bdf906f801f59acefc49094361a530b823a73ab8`

Previously reviewed implementation: `6345de8c488f3a7f2193a0fc8c067067334278ef`

Candidate: `TapeoutMinerPool/2.3.0`, package `2.3.0-rc.1`

Review date: 31 October 2026 UTC

Reviewer: ChatGPT, GPT-6 Astra Pro, with parallel contract, tooling and evidence review tracks

Authorization: Owner-authorized internal review of an unreleased candidate. Historical mainnet canary activity used owner-controlled assets and is not validation of this candidate.

Deliverables: This complete English report, a PDF edition, a machine-readable summary and a separate evidence archive. Source references resolve to the frozen target; evidence paths below are relative to the archive root unless stated otherwise.

1. Executive conclusion

The exact frozen candidate can initialize one shared distributor and operate both custody pools after the required ownership renunciations. Fresh execution supports the complete local CLI lifecycle, both directions of staggered startup on a pinned real-protocol fork, late receipt delivery, independent claims, transfer of unequal unpaid histories, and additional income accruing to the new holder. No intrinsic circular setup dependency, cross-pool accounting collision, duplicate receipt delivery, or new High/Medium failure of the honest, fully configured two-pool route was confirmed.

This is a qualified functional conclusion, **not production custody approval**. Two inherited Medium operational-verification weaknesses remain: an incomplete manifest can receive an unqualified `MATCHES` status without checking the whole pair or its actual bindings, and the runtime comparator masks each immutable occurrence independently. Both were reproduced against the actual frozen SDK. The first now has an additional optional pool to omit; the second applies to both pools and the enlarged distributor runtime. Neither reproduction demonstrates an unprivileged attack against an already correctly deployed and bound genuine system.

The contracts deliberately rely on correct setup. Membership checks do not enforce reciprocal bindings to one distributor, and `allocationFinalized` does not prove that the full issued supply was funded and allocated. A fresh genuine-code counterexample finalized a second empty distributor and bound the two pools to different distributors; both could open custody. The full-manifest verifier rejected divergent binding fixtures, and the local CLI rejected a manifest selecting a second distributor; direct contract calls permit the split topology. These are confirmed setup boundaries, not evidence that the supported lifecycle is unable to run.

The shared receipt pair also expands the impact of an inherited irreversible allocation or receiver mistake: that receipt carries rights in **both** pools. The historical owner acceptance of specified SALUS operational risks is relevant context, but it does not establish correct new launch inputs, approve this candidate, or close the independent identity and compiler gates.

The pinned compiler remains in the affected version/settings class for the official Low-severity SOL-2026-7 advisory. Actual miscompilation of this build was neither demonstrated nor excluded. A reproducible build and successful tests do not resolve that inherited release-engineering gate. See section 8.3 and sources S15-S16.

1.1 What was established, and what remains open

Table 1. Review outcome and open boundaries

Question	Review result
One distributor, one receipt inventory, two functioning pools	Supported by source analysis, the fresh local CLI lifecycle, independent local proofs and real-protocol fork tests.
Independent claims and transfer histories	Supported by literal asymmetric expected values and real-protocol tests in both startup directions.
Ongoing economic isolation	No new peer-pool call in admission, harvest, receipt claim or role release. Shared components and common upstream dependencies remain shared failure domains.
Complete topology and runtime identity from a generic `MATCHES` result	Not established. T23-01 and T23-02 remain open.
Actual launch allocation, recipients, miner routing, capacity and current dependency authority	Not verified; the review used controlled fixtures and a pinned historical fork.
New confirmed High/Medium production-contract defect	None identified within the reviewed and executed scope. This does not exclude undiscovered defects.
Deployment consideration	Resolve the inherited verification/compiler gates, then verify the actual launch inputs at every irreversible boundary. No production transaction is authorized by this report.

2. Scope, provenance and evidence discipline

2.1 Exact revisions and changed surface

The fresh source checkout was detached at the requested target. The observed `origin/main` at acquisition was also `02ef1f7da17683b482b97b28464346bc2cb9db61`; no newer revision was substituted. The target's immediate parent is exactly `bdf906f801f59acefc49094361a530b823a73ab8`.

The immediate baseline and prior audited commit are different Git trees. Independent comparison found **no changes** between them under `src/`, `sdk/`, `cli/`, `script/`, `scripts/`, or in `foundry.toml`, `package.json` and `package-lock.json`. The intervening changes add historical reviews, evidence, review tests and operating-risk documentation. Their existence does not constitute a production remediation. The baseline-to-target diff changes 101 files and was inspected as the current change boundary. Evidence: `evidence/provenance-source-bytecode.json` and the initial/final source manifests.

There are five production deployments using three implementations: NAND receipt, LATCH receipt, Optimal Pool, Regular Pool and the shared distributor. The material implementation changes are the distributor's two-member constructor and views, validation of both members and histories, and exclusion of either pool as a beneficiary. Pool binding now asks `isPool(address(this))` instead of comparing the removed single-pool getter. The pool version changes to 2.3.0. SDK schemas, selection, deployment, gas assumptions, artifacts and runners change accordingly. [S1-S8]

After stripping comments and whitespace, the Pool token stream matches the baseline after only the version and membership-expression substitutions. `TapeoutEntitlementReceipt.sol`, `PoolMath.sol`, `IMining.sol`, `sdk/math.ts` and `docs/ACCOUNTING.md` are byte-identical. This establishes unchanged

per-pool accounting source; it does not establish unchanged execution dependencies or byte-identical Pool code.

Table 2. Source and bytecode change classification

Artifact	Fresh comparison with retained 2.2 build	Current runtime size
EntitlementReceipt	Creation/runtime exactly identical; ABI unchanged	8,743 bytes
MinerPool	Executable change; runtime grows by 2 bytes; ABI unchanged	23,170 bytes
ReceiptDistributor	Constructor/runtime and ABI changes	8,137 bytes
PoolMath	Creation/runtime exactly identical	58 bytes
IReceiptDistributor	ABI changes; interface has no executable bytecode	0 bytes
Recoverable canary pool	Separate executable change, runtime grows by 2 bytes	24,441 bytes
CanaryParticipant	Trailing compiler metadata changes only	3,410 bytes

The target was freshly compiled. The prior comparison uses the retained earlier fresh build, after independently establishing implementation equality between 6345de8 and the immediate baseline; it is not described as a new compilation of the baseline. All six generated ABI/bytecode entries were independently regenerated into separate files. The complete `sdk/abi.ts` and `sdk/bytecode.json` matched exactly, with SHA-256 values `258dadb272aeea19a4c882d8897d945cf3c4cb6adc8ca4507ca16ee05a901174` and `393e56bd8b202dc94b2b175d1c3a97eba9f384c91fdc8e1963e9c0b9d497a37f`. This proves artifact correspondence to the pinned build, not compiler correctness.

2.2 Prior reports and SALUS context

The actual earlier Astra review was located at the repository root as `INTERNAL_GROUPMINE_2_2_REVIEW_6345de8_GPT6_ASTRA_PRO.pdf`. It has 41 pages. Although its PDF bytes differ from the previously delivered edition because the repository copy has stripped metadata, extracted text from every page is identical. No nonexistent repository Markdown report was assumed. The actual Fable review is `docs/INTERNAL_GROUPMINE_2_2_REVIEW_6345de8_FABLE51.md`. Both retain their original target and evidence limits. [S10-S12]

The original SALUS assessment targets `c3cb93375737de7ec63baa9f390edd0c15a5c7a4`, not this candidate; its reported findings remain Pending in the supplied assessment. The implementer's remediation descriptions are not SALUS closure. Current assumptions record owner acceptance of original SALUS #1, #4, #5 and #6 under operating constraints. That acceptance is policy evidence, not proof that actual deployments, allocations or custody satisfy those constraints. Relevant inherited code and finality regressions were re-executed; unrelated historical suites were not automatically repeated. [S11-S13]

2.3 Evidence classes and execution controls

Table 3. Evidence classifications

Label	Meaning in this report
EXECUTED	This review actually ran the stated command or test and retained its result. A passing limitation-reproduction test confirms the limitation.
SOURCE	Direct reasoning from the frozen implementation, tests, configuration or artifact comparison. It is not a runtime result.
AUTHOR-RECORDED	A developer or previous reviewer recorded execution. It was inspected but is not counted as newly executed here.
NOT EXECUTED / UNVERIFIED	A specific unavailable, interrupted, deliberately out-of-scope or missing-input check. It is neither PASS nor proof of a candidate failure.

Tests ran in an isolated archive of the frozen source, with new proofs under `test/independent/` and evidence outside the pristine checkout. Locked npm dependencies were installed with lifecycle scripts disabled. Executable runners and the three numeric FFI model programs were inspected before use. Production wallet files and credential stores were not accessed. The controlled environment did not inherit wallet-bearing variables.

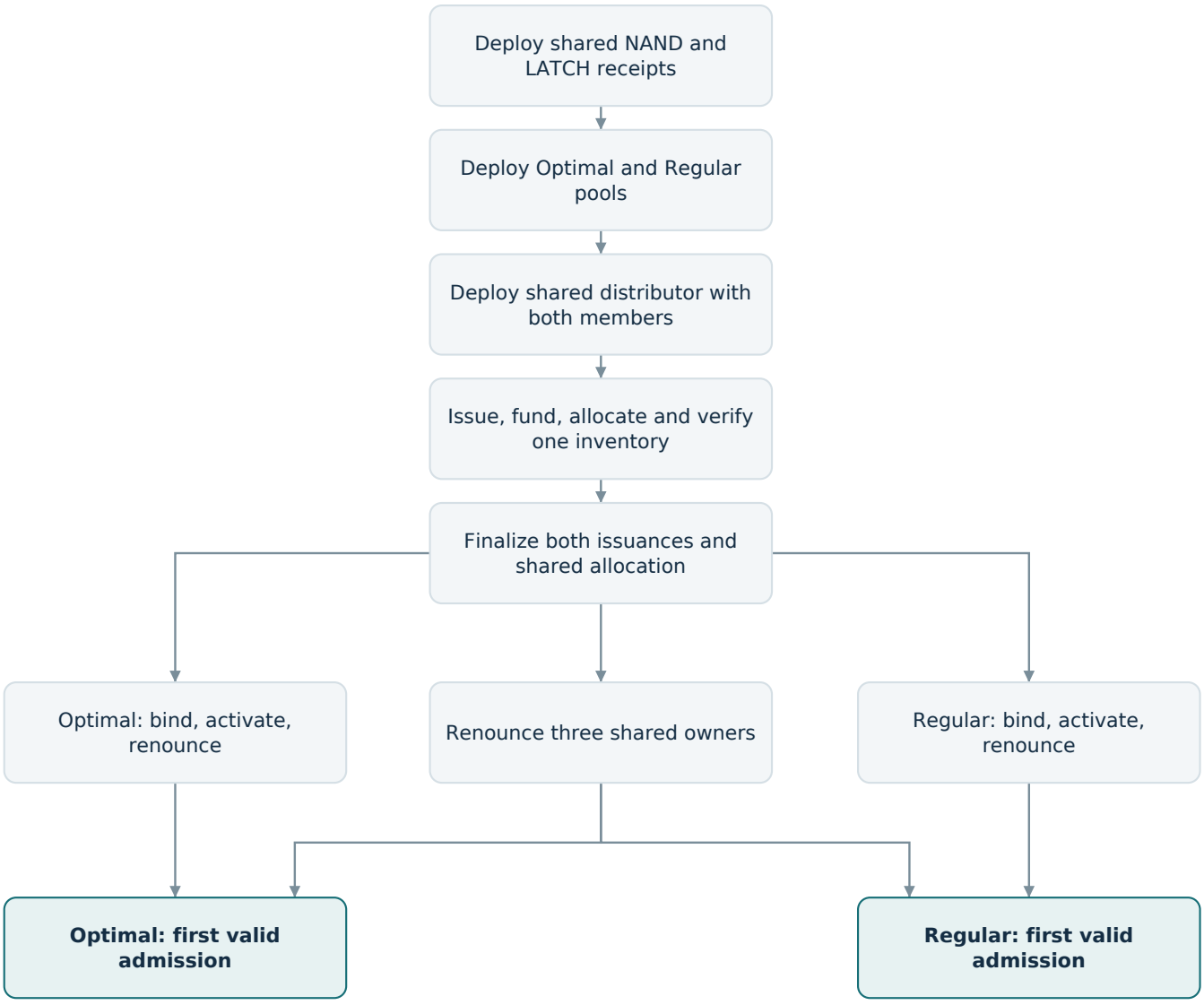
Real-protocol state came through a loopback Anvil endpoint verified by client type, chain, block number and block hash. Chain ID 56 alone was never accepted as proof of isolation. The upstream public endpoint was used for state reads only. Local impersonation and deterministic synthetic Safe test-key signatures occurred solely on isolated forks. No public-chain transaction was signed or broadcast, and no upstream protocol code/storage was patched in the real-protocol lifecycle tests.

The pristine checkout and original implementation/test inputs were retained unchanged. Final preservation results and hashes accompany the evidence. New reproduction files, build output and generated test reports belong to the execution copy only.

3. Initialization: executable order and irreversible boundaries

3.1 Dependency diagram

Figure 1. Shared setup and independent custody branches for genuine implementations.



Each custody branch requires shared finality and its own completed setup.

The diagram shows one permitted contract order. The correction-preserving sequence below finalizes issuer-held supplies before funding. Binding may occur before allocation finalization, and shared renunciations may occur after either pool's setup, provided all owner-only work is complete. **There is no edge requiring one pool's activation or admission before the other pool can open custody.** Both pool contracts must already exist at distributor construction. Delivery becomes available at allocation finalization and need not wait for every renunciation or admission. [S1:298-328, 746-754; S2:91-143, 203-227]

3.2 Working sequence

1. Approve the complete intended system: chain, exact build, protocol addresses, five deployment roles, owners, both recipient sets, fixed depositor whitelist, intended miner-to-pool routing, receipt supplies and original contribution/allocation JSON. These are launch inputs, not facts established by a sample manifest.
2. Deploy both genuine receipt collections with their correct categories. Deploy both genuine pools using the same receipt pair, BEM, Mining and miner collection, while preserving each pool's approved owner, recipients and depositor configuration.
3. Deploy one genuine distributor with the exact ordered Optimal and Regular addresses. Check both getters, `isPool` membership, receipt identities and complete runtime provenance before any irreversible funding or binding.
4. Issue one inventory. Issuing first to the issuer preserves the limited top-ID retraction window. Direct issuance to the distributor is supported and was tested, but should not be confused with retaining that correction ability. Correct counts before transferring required IDs or finalizing issuance.
5. Finalize both issuances with the approved counts. Fund the distributor once with the complete issued inventory. Import one allocation set. Independently compare every category/ID, beneficiary and range with the approved original JSON and confirm actual receiver usability. The second pool does not need a second copy of any receipt.
6. Verify full issued-supply coverage, accepted inventory, allocations and current holdings; delivered counts must still be zero. Then finalize the allocation. This freezes allocations and opens permissionless delivery immediately. Subsequent launch rechecks must reconcile legitimate deliveries and transfers as well as remaining inventory. `allocated == inventory` alone is insufficient for full-round assurance.
7. Bind this distributor once in each pool and activate each using the exact finalized supplies. A pool may remain unbound and inactive while the other proceeds. Its owner must remain able to perform both setup calls. A zero supply for one category is supported; the combined supply must be nonzero and admissions cannot carry burn in an unsupported category.
8. Complete metadata and other owner-only setup, then call standard `renounceOwnership()` on both receipts, the distributor and each pool that is ready. Genuine two-step ownership clears the pending owner on renunciation. Ownership transferred to `0x...dEaD` is not zero ownership and does not satisfy the custody gate.
9. Immediately before the first admission to each pool, verify that pool's activation, owner zero, exact binding and denominator values, plus final/ownerless shared components. Account for unexpected pre-existing NFT or BEM balances. Have the approved depositor use atomic `depositMiners` for the reviewed eligible miners.
10. Continue delivery, per-pool harvest, receipt claims, A/B releases and receipt transfers. The delayed pool can later bind, activate, renounce and admit without reopening shared ownership. Pool choice must remain explicit for every pool-scoped operation.

These dependencies and supported variants were exercised through the unmodified local CLI, independent local contract proofs and the pinned fork scenarios. The CLI preserves issuer-held correction before funding, while some proofs issue directly to the distributor. The tests establish executable routes with reviewed fixtures; they do not validate a future original contribution list or prove all possible setup orders safe.

3.3 Why funding does not deadlock

The new inventory check reads `claimed(category, id)` in both pools. In the genuine Pool this function is an unconditional storage view at lines 521-522; it does not require `active`, an open custody latch, a registered miner or a successful Mining call. Fresh zero histories therefore pass while both pools are unbound, inactive, owner-held and empty. The OR expression can short-circuit on nonzero Optimal history; the normal zero-history path reads both pools. [S1:521-522; S2:297-307]

Activation requires the bound distributor's finalized allocation and finalized receipt supplies. First admission then requires that pool and the three shared contracts to be ownerless. It does not query the peer pool's owner, activation or registry. The independent local test started Regular while Optimal remained unbound and owner-held, then completed Optimal after shared renunciation. Both directions also passed on the real-protocol fork.

An admission failure is atomic. The new first-admission proof deliberately failed after entering the admission path and verified restoration of the custody latch, NFT holder, registry, composition, BEM balances and carried-income accounting. A temporary failed transaction does not partially open custody. A successful permanent admission, however, has no production withdrawal or migration path. [S1:625-699, 706-754]

3.4 Irreversible steps

Issuance finalization ends mint/burn correction. Funding removes issuer-held top-ID retraction for those IDs and provides no general distributor return route. Allocation finalization freezes beneficiaries and opens delivery. Pool binding is one-time. Activation fixes the share denominators. Renunciation removes the relevant setup authority. Admission takes permanent custody.

Premature renunciation of either shared receipt issuer or the distributor can prevent completion for both pools. Premature renunciation of a pool can strand that pool's setup even if its peer operates. Redeploying fresh contracts does not retrieve, cancel or invalidate receipts or assets already held by abandoned contracts. The historical owner acceptance must be applied to the actual stage and expanded shared impact, not described as a general recovery mechanism.

4. Independent economic reasoning and observed results

4.1 Separate state, common receipt identity

Each pool owns its own ledger and per-category/per-ID claimed-history mappings. The effective key is `(pool address, receipt category, receipt ID)` because the storage belongs to a distinct contract address. NAND #1 and LATCH #1 are different keys, and each key has a separately stored history in Optimal and Regular. The distributor neither merges BEM balances nor claims rewards on behalf of users. [S1:418-440, 521-552, 817-839; S2:220-227]

The holder check uses actual receipt ownership, not transfer approval. Once a receipt moves, its previous holder loses future claim authority; its historical paid amount remains in each pool. An approved ERC-1155 operator cannot claim solely because it has transfer approval. Source reasoning, existing claim tests, independent local proofs and the real-protocol transfer tests agree on this behavior.

For each pool separately, recognized total `T` yields role entitlements `A = floor(T/84)`, `B = floor(T/84)` and `C = floor(2*T/84)`. C remains fixed to `0x000000000000000000000000000000000000dEaD`; A/B release settles that pool's C backlog atomically. Participant accounting uses the unchanged scaled accumulator with `H = 80 * 10^18` and `K = 84 * 10^18`. For realized carry `r` of one incoming miner with NAND:LATCH burn `n:l`, the contribution is `uN = floor(r*H*n/(n+l))`, `uL = r*H-uN`. Subsequent ordinary income uses that pool's aggregate composition/checkpoint. Per-receipt cumulative income is obtained from the category accumulator and fixed share count, less the paid history for that pool and ID. [S1; S3; S9]

This preserves separate rounding. It does not apply another 80:1:1:2 fee to the sum of both pools, and it does not use one pool's material ratio for the other's revenue. Exact integer floors at the implementation's intermediate steps remain relevant to the inherited one-raw-unit caveat discussed in section 8.1.

4.2 Literal asymmetric oracle

The new local proof used 10 NAND IDs and 5 LATCH IDs, distinct pool compositions and incomes, and independently derived literal expectations. There was zero carried balance and no intervening accrual in the exact example.

Table 4. Independently derived asymmetric economic example

Quantity	Optimal, burn 3:1	Regular, burn 1:3
Recognized income	1,050 BEM	2,100 BEM
NAND category income	750 BEM	500 BEM
LATCH category income	250 BEM	1,500 BEM
A	12.5 BEM	25 BEM
B	12.5 BEM	25 BEM
C	25 BEM	50 BEM
One NAND receipt	75 BEM	50 BEM
One LATCH receipt	50 BEM	300 BEM
One receipt of each category	**125 BEM**	**350 BEM**

The receipt-claim assertions matched the derived per-receipt amounts after income accrued while the distributor still held the receipts; later role-release assertions provided additional ledger checks. One delivery then made the holder eligible in both pools. Reversing claim order preserved the same respective payments when no new income arrived.

The seller next claimed only Optimal and transferred both receipts to a buyer. The buyer received the unpaid **350 BEM from Regular and zero previously paid Optimal income**. Additional income of 420 BEM in Optimal and 840 BEM in Regular produced **50 BEM and 140 BEM** of new rights for that buyer. Claims updated their own category/ID histories; unrelated IDs and peer-pool histories remained independent. Evidence: `contracts/IndependentTwoPoolContractReview.t.sol`, named tests in `logs/two-pool-local-corrected-result.json`.

4.3 Carry, tiny values and conservation

A separate proof introduced 1,050 BEM of carried income only into Regular with a NAND-only incoming miner. Its participant portion was 1,000 BEM of NAND income despite Regular's prior 1:3 composition; Optimal remained unchanged. This directly checks the SALUS carry remediation in the presence of two pools, rather than relying only on unchanged source.

With 42 raw BEM units recognized by each pool, each A entitlement is zero even though `floor((42+42)/84)` is one. Each C entitlement is one raw unit. Participant income is 40 raw units per pool; with the tested compositions/supplies, one NAND plus one LATCH receives five raw units from Optimal and seven from Regular. A shared sink wallet may receive the sum of payments, but its global wallet balance is not either pool's private `C` history. Zero-income and zero-LATCH-supply cases also passed.

Conservation was asserted per pool before combined totals: paid amounts plus remaining recognized liabilities and rounding residue reconcile to recognized income, and actual token balance equals recognized unpaid value plus any unrecognized inflow. Per-ID histories prevent payment above the cumulative entitlement. A/B/C payments were attributed to their originating pool, including cases with shared destination addresses. These checks are bounded to the tested sequences and exact frozen arithmetic; they are not a proof over every possible upstream token behavior.

4.4 Real-protocol compatibility

The fork was pinned to BSC block **128,713,128**, hash `0x760f050b7ec66ced2a2745918d7492650150f9d71d3159259412996ebe65081f`. It used genuine upstream state, miner **12733** for Optimal and **13948** for Regular. The first miner is NAND-only; the second has NAND 2/LATCH 64 burn. New production deployments used the exact Paris artifacts. The local execution host used Cancun support; this did not recompile the deployed candidate for Cancun.

Two new tests exercised Optimal-first and Regular-first operation with the delayed peer unbound, inactive and owner-held. They then bound the delayed pool after shared renunciation, accumulated distinct actual income, delivered late, tested unequal history transfer and subsequent income, and reconciled separate ledgers. No upstream code or storage was replaced. Local NFT transfers used fork-only impersonation.

Table 5. Fresh pinned-fork observations

Observed value at delivery	Optimal	Regular
Total recognized income, raw 8-decimal BEM	20,143,940,127	1,080,898,513
NAND per-receipt income, raw units	1,918,470,488	3,119,476
LATCH per-receipt income, raw units	0	199,646,478

The two scenario reports have different realized carried amounts because admission timing differs; this must not be presented as universal admission-order independence. The same final totals in these particular scenarios are observations, not a guarantee of identical upstream mint/payment outcomes under different transactions. Files `logs/fresh-review-two-pool-optimal-first.json` and `logs/fresh-review-two-pool-regular-first.json` retain the exact carry and oracle observations.

5. Distribution and failure isolation

5.1 External-call map

Table 6. Cross-pool external-call and failure map

Phase	Dependency introduced or relevant to the pair	Consequence
Distributor construction	Both member code-presence checks and receipt/protocol getters	Incompatible members reject construction; getter compatibility is not runtime authentication.
Receipt funding or direct issuance into distributor	Both pools' per-ID `claimed` views	Consumed history in either member rejects fresh inventory. A mimic getter can prevent funding. Genuine inactive pools return zero normally.
Allocation	Fixed receipt holdings and beneficiary exclusions	Neither member pool can receive an allocation. No second inventory is created.
Pool binding/activation	That pool's selected distributor and shared receipts	No reciprocal peer binding check. Activation requires shared allocation finality.
First admission	Fresh owner reads for that pool and the three shared contracts; finality already established at activation	Shared owner/finality mistakes affect both; peer activation/ownership is not a prerequisite.
Finalized delivery	Shared receipt transfer and beneficiary callback	No call to either pool or Mining unless the beneficiary itself chooses one in its callback.
Pool harvest/admission	That pool's registry and common Mining/token/NFT dependencies	No direct peer-pool call. Common external failures may still affect both independently.
Receipt claim / role release	That pool's ledger and BEM transfer	Claim does not harvest. A pool-local failure does not read or mutate the other ledger.
Full-manifest CLI write preflight	Both pools' identity/configuration getters	Malformed identity can stop all writes; ordinary Mining/economic failure is not queried.

5.2 Once-only delivery and callbacks

Finalization prevents later inventory or allocation mutation. Delivery advances the wallet cursors and counters before transferring receipts, under a reentrancy guard. Repeated delivery cannot mint or reassign the already delivered IDs. The distributor has no owner sweep, arbitrary recipient override, ERC-1155 approval route, or reward-claim forwarding function. Permissionless `deliver` still sends only to the recorded beneficiary. [S2:203-227, 314-336]

The receipt implementation updates holder accounting before invoking acceptance callbacks. A receiver can therefore legitimately claim its newly owned rights from either active pool during delivery. The independent callback proof claimed both pools and attempted reentrant redelivery: the proper claims succeeded and redelivery did not duplicate receipts. A second proof rejected the later LATCH callback after the earlier NAND transfer and nested claims; the entire transaction

restored both transfers, both pools' claims, BEM balances, cursors and counters. This atomic rollback is the expected behavior, not a lost-delivery state. [S4:131-155]

A beneficiary that cannot accept a receipt, or accepts it but has no usable claim/transfer mechanism, can still strand its own rights. ERC-165/receiver interface support alone does not establish usability. A callback that insists on claiming an inactive peer pool can voluntarily make delivery depend on that pool and revert; this is source reasoning about receiver behavior, not a reproduced defect in the distributor. Actual beneficiary code and control must be reviewed before finalization.

5.3 History rejection and reachability

Freshly executed existing tests reject inventory with consumed history in Optimal or Regular, including Regular-only history. The two-pool test obtains real paid history through ordinary claims in genuine running pools, then attempts funding a second, unbound genuine distributor. IDs paid in either pool are rejected, while an unused ID is accepted. No storage injection is used in that test. The original finalized distributor independently rejects all returns. In the intended initial route no pool can activate until allocation finalization, and after finalization that distributor's funding hooks are closed. The test therefore proves the defense on a later distributor without implying that ordinary initial funding requires a pool to have operated already.

5.4 Failure experiments and their limits

The new local tests injected a key-specific Regular Mining failure: its harvest call returned zero received and one failed miner, and its next admission reverted during strict settlement. Delivery and Optimal operations remained functional. A sender-specific mock BEM failure at the sink rolled back Regular's A-and-C release, while Optimal release and both receipt claims still worked; B was not exercised in that selective-failure case. These establish isolation of the tested control flow; the selectively failing mocks are not claimed to be the behavior of the immutable genuine upstream contracts.

A getter-compatible second-pool mimic could pass distributor construction and later fail its history view, preventing funding. The failed funding rolled back. After funding/finalization, making that mimic's `claimed` view fail did not block delivery because delivery does not query it. Genuine Pool `claimed()` is a storage read and does not acquire such failure behavior merely because Mining fails or the pool is inactive.

Common BEM, Mining and NFT dependencies remain common risks. A global BEM transfer failure can affect both pools, and an upstream failure can independently obstruct each pool's admission or harvest. Independent internal ledgers do not promise independent availability from shared external contracts.

6. Findings and confirmed setup boundaries

Severity describes the consequence under the stated actor and preconditions. A Medium setup-verification finding does not imply permissionless exploitation of a genuine ownerless deployment. Confidence refers to the demonstrated observation; it must not be extended to an unexecuted malicious-runtime exploit.

Table 7. Categorized findings register

ID	Category and severity	Change classification	Status
T23-01	SDK/CLI - Medium	Inherited, expanded to two optional pool roles	Reproduced; open
T23-02	SDK/CLI - Medium, conditional setup risk	Inherited comparator; enlarged distributor immutable surface	Predicate weakness reproduced; asset exploit not executed
T23-03	SDK/launch input - Low	Inherited recipient-note behavior, two recipient sets	Reproduced; approved roles still need proof
SETUP-01	Production setup boundary	Inherited full-round limitation plus new reciprocal-binding choice	Genuine-code counterexample executed
SETUP-02	Production authenticity boundary	Inherited getter trust, second member added	Mimic behavior executed; no genuine runtime failure implied
EX23-01	Compiler dependency - upstream Low	Inherited affected compiler/settings	Exposure class confirmed; actual miscompilation unverified
EVID-01	Evidence - Informational	Missing raw logs in new author package	Confirmed; structured results remain available

6.1 T23-01 - Partial manifests can certify an incomplete pair

Severity: Medium. **Confidence:** High. **Actor/preconditions:** A mistaken or malicious setup/manifest provider, or an operator treating `MATCHES` as full-system validation. This does not let an outsider replace a genuine pool's already fixed binding.

Source: `sdk/config.ts:45-53` makes pool and distributor addresses optional for staged setup. `sdk/deployment.ts:71-76` skips an omitted pool. Lines 109-139 read each pool's actual distributor only inside `if (addresses.distributor)`. The CLI's common write preflight invokes this checker at `cli/pool.ts:299-305`. [S5-S6]

Fresh reproduction: The independent SDK probe called the actual frozen `checkDeployment` with both pools bound to a different distributor in the fixture. Omitting the distributor from the manifest returned `MATCHES` and performed **zero actual binding reads**. A manifest containing only the two receipts and Optimal also returned `MATCHES`, with zero Regular reads. With a complete manifest, the same incorrect binding returned `MISMATCH`. Swapped roles and an unintended third pool were also correctly rejected when all addresses were supplied.

Impact: Partial setup inspection is useful, but its unqualified success can be mistaken for complete operational verification before irreversible binding, funding or admission. The second optional pool expands this assurance gap. Documentation that every write verifies both pools must state the manifest-completeness condition. The supported complete-manifest route works; the output contract is too broad for partial input.

Minimal remediation: Introduce phase-aware validation. Return `PARTIAL` or `INCOMPLETE` when required deployment roles are absent; require the full five-address operational manifest before actions that rely on complete identity. Read and authenticate each supplied pool's actual bound distributor regardless of whether a manifest field is present. Explicitly permit a known delayed pool to remain unbound during its legitimate setup phase, rather than blocking staggered startup. Include which roles and bindings were checked in the status.

An additional executed scope check is relevant: allocation verification returned `FINALIZED_AND_MATCHES` for a fixture with Regular bound elsewhere, while full `checkDeployment` returned `MISMATCH`. Allocation verification does not read reverse pool bindings. This is not counted as a separate Medium defect because the normal full-manifest finalization command performs both checks. A standalone allocation success must not be cited as proof of complete topology.

6.2 T23-02 - Masked immutable occurrences are not fully authenticated

Severity: Medium under malicious or untrusted setup-code provenance. **Confidence:** High for the comparator weakness. **Actor/preconditions:** Someone able to supply a deployment/address whose bytecode is then trusted through this predicate. No post-deployment mutation of genuine immutable code is assumed.

Source: `sdk/deployment.ts:17-28` zeroes every immutable range independently before comparing runtime templates. `scripts/export-abi.mjs:23` flattens the compiler's grouped immutable references into ranges. Getter checks then observe only the particular occurrences used by those getters. [S5, S8]

Fresh reproduction: The actual frozen `compareRuntime` accepted byte strings whose immutable slots were independently changed: **65 Pool occurrences** and **16 Distributor occurrences**. Fresh compiler artifacts identify five Pool groups with 20, 16, 14, 7 and 8 occurrences, and four Distributor groups with four occurrences each. The exporter discards the relation that each occurrence in a group should represent the same constructor value.

Execution boundary: This was a pure SDK-predicate test using byte strings plus artifact/source analysis. No altered-runtime contract was deployed, no operational malicious bytecode path was executed, and no asset loss was demonstrated. The result establishes that matching template code plus getter values is not a complete proof that every operational immutable occurrence contains the intended value.

Impact: Runtime identity is part of the setup trust boundary because the contracts themselves do not authenticate Pool/Distributor runtime. A checker used to replace trusted deployment provenance must validate the complete instantiated runtime. Repeating the same incomplete comparator for two pools does not close the gap.

Minimal remediation: Retain compiler immutable grouping and verified constructor/build provenance; construct the exact expected runtime with every occurrence filled consistently and compare all bytes. Independently verify constructor arguments, deployment transaction/initcode, relevant storage initialization and current approved mutable roles. A fully authenticated deterministic deployment can provide equivalent evidence. Regression tests should alter one

operational occurrence while leaving getter occurrences unchanged and require rejection. Such a remediation belongs to a new candidate; none was applied here.

6.3 T23-03 - Recipient mismatch is only a note

Severity: Low. **Confidence:** High. **Actor/preconditions:** Incorrect initial recipient configuration or unexplained current role state; an operator relies solely on status. Recipient roles can legitimately move through their own authorization, so a current difference is not automatically malicious.

Source and reproduction: `sdk/deployment.ts:95-98` adds recipient A/B differences to `notes`, not `problems`. The actual SDK returned `MATCHES` with a note for a wrong Regular recipient under a complete system manifest. The message suggesting a role may have moved does not prove that a legitimate move occurred. [S5]

Impact and minimal remediation: A wrong fee recipient can survive a superficially successful launch check. Strictly validate initial recipients against authenticated constructor inputs; after a legitimate migration, compare with an approved current-role record and supporting event/transaction history. This is not a separate new release blocker where independent role evidence exists. Both recipient sets remain unverified for an actual launch in this review.

6.4 SETUP-01 - Membership and finality do not establish the whole intended topology

Classification: Confirmed production setup boundary; conditional irreversible-setup risk. **Confidence:** High. **Actor:** A mistaken or malicious setup owner before one-time binding. This is not an unprivileged replacement of an honest completed deployment.

Source: Pool binding checks membership and receipt getters but not the peer's binding (`TapeoutMinerPool.sol:298-328`). Distributor finalization checks allocation count against accepted inventory, not total issued supply (`TapeoutReceiptDistributor.sol:203-214`). Each pool's custody gate trusts its own distributor (`TapeoutMinerPool.sol:746-754`). [S1-S2]

Executed counterexample: Deploy two genuine distributors D1 and D2 with the same Optimal/Regular pair. Fund and fully allocate D1. Finalize D2 with zero inventory and zero allocation. Bind Optimal to D1 and Regular to D2, activate and renounce both, then admit miners. Both pools can operate. Receipts delivered by D1 still earn correctly in both because the reward ledger uses receipt ownership, not the distributor address.

The counterexample violates the intended single-distributor topology without causing an intrinsic accounting collision. It demonstrates exactly why owner-zero, membership and allocation-finalized flags are insufficient evidence of a reviewed full round. A third genuine pool cannot bind the intended two-member distributor, as tested, but a setup owner can choose a different genuine pair/distributor before binding. Full-manifest SDK verification catches the reproduced divergence and unintended third-role assignment.

Minimal treatment: Authenticate all five intended deployments, both role addresses and reciprocal bindings; reconcile every issued category/ID with the one approved allocation and current delivery state. Do not require duplicate inventory. Keep full-round verification distinct from count equality and original contribution truth. Stronger reciprocal on-chain enforcement would be a design change; this review does not require it to conclude the documented trusted-setup route is executable.

6.5 SETUP-02 - Getter compatibility remains a trust boundary

Classification: Confirmed authenticity boundary, enlarged by the second member. **Confidence:** High. **Source:** Distributor constructor lines 91-110 and inventory validation lines 297-307; Pool binding/finality checks in S1.

The distributor verifies code presence, distinct members, matching receipt addresses and matching BEM/Mining/collection getters. It does not prove either member uses the reviewed Pool implementation. Conversely, a Pool authenticates the exact receipt runtime/category but trusts the selected distributor's membership, receipt, finality and owner views. Fresh retained regressions also confirmed a mimic distributor passing the on-chain gate.

The new mimic-pool test proves that a getter-compatible member can selectively block funding; it does not establish a failure path in genuine `claimed()`. The prevention is authenticating every Pool and Distributor before funding/binding, with the limitations in T23-01/T23-02 resolved or independently covered. Treating an attacker-controlled setup owner as trusted would simply assume away the condition; the report keeps that assumption explicit.

6.6 EVID-01 - Referenced developer raw logs are absent

Severity: Informational. **Confidence:** High. All 18 raw step-log paths referenced in the new developer run manifest/README are absent from the frozen evidence tree at the advertised copied location. Structured JSON/TAP/text results are present. The independent source fingerprint matches the target's relevant code tree, which supports correspondence of inputs but not every execution claim.

This is missing reproducibility evidence, not evidence of fabricated testing. Retain the raw stdout/stderr, per-file hashes and failed attempts alongside future summaries. Fresh review execution creates new evidence; it cannot retroactively restore the author's original logs. See `evidence/evidence-review-notes.md` and `evidence/provenance-source-bytecode.json`.

7. Operational correctness, gas and tool behavior

7.1 Pool selection and workflow

Pool-scoped commands require an explicit `--pool optimal` or `--pool regular`. The parser rejects missing/invalid values and duplicate selector flags instead of silently defaulting. Actual offline CLI probes exercised these cases. Shared commands such as system validation, receipt issuance and distributor allocation do not require a meaningless pool selector. Old single-pool schemas fail clearly. SDK builders preserve distinct owners, recipients and depositor configurations while sharing protocol/receipt addresses. [S6-S7]

The 20 new tooling probes also checked nine planned transactions for each role, including separate A and B releases, canonical duplicate-ID handling without combining NAND/LATCH categories, wrong chain, incorrect whitelist/owner/protocol, role swaps and delayed unbound peers. Read-version isolation accepts the current production/canary layouts and rejects interpreting the older 2.2 layout as the new version. Existing SDK/audit TypeScript suites were freshly run as well.

The full unmodified local CLI lifecycle completed **79 checks across 90 steps**, covering deployment of both pools and one distributor, one allocation round, finality, explicit pool operations and

independent reconciliation. This used the local mock protocol. It complements, rather than replaces, the genuine-protocol fork tests.

Full-manifest write preflight reads both pools' identity pointers and optional owner/whitelist/recipient configuration. It does not query Mining, harvest state, claimable rewards, active flags or role-payment balances. Therefore a normal genuine Mining failure in Regular does not itself make Optimal's identity preflight fail. A malformed getter or persistent topology mismatch can stop all full-manifest writes; this is an identity failure, not shared economic accounting.

Offline plans bind chain and target. Local execution guards require an HTTP loopback endpoint without URL credentials and an Anvil client, not merely chain ID 56. This review did not invoke live deployment or recovery commands.

7.2 Cold transaction bounds and Safe execution

The additional zero-history read is relevant to issue-directly-to-distributor and funding gas. The updated SDK budgets 56,000 gas per issued ID and 44,000 per funded ID, using an application planning budget of **12,582,912 gas**, three quarters of 16,777,216. The independent planner probe obtained maximum planned batches of **222 issue IDs** and **283 fund IDs**. These are planner outputs, not a proof of unlimited execution capacity. [S7; S17]

The existing GasModel suite passed **11/11** tests. Its isolated-call configuration and explicit transaction-separation assertion were inspected. It checks direct issuance, funding, deployment, allocation/revocation, claim and fragmented delivery bounds. Reported whole-test gas can exceed a transaction cap because a test contains many calls/deployments; it is not quoted as one production transaction's gas. Fresh JSON capture did not retain console gas lines, so this report uses the assertions as bound evidence and the following Safe receipts as concrete fresh transaction measurements.

The unmodified Safe fork runner passed **294 checks**, with **28 Safe transactions**, **47 total local fork transactions** and **67 synthetic test-key signatures**. All were local to the verified fork. The script used canonical Safe 1.4.1/1.3.0 contracts at the pinned state; no live wallet credential or public-chain signature was used.

Table 8. Fresh local Safe transaction measurements

Fresh Safe setup transaction	Estimated gas	Actual gas used	Planned bound
First issuance batch, two calls of 100 NAND IDs	9,828,900	9,454,076	11,934,896
First funding batch, two calls of 125 NAND IDs	11,020,622	9,392,546	11,990,373

Across the 28 Safe transactions, the maximum estimate was **11,098,058**, maximum planned bound **12,004,688**, and smallest estimate headroom **148,815**. The zero-history funding path checks both pool histories. These measurements support the tested batch sizes; Safe configuration, signatures, transaction state and actual beneficiary code can affect other cases.

Safe coverage limit: It binds, activates and renounces both pools and tests shared issuance/funding. Its miner admission, reward payment, transfer-following claim and deep reconciliation assertions target **Optimal only**; Regular remains initialized and empty. It is not counted as a second asymmetric economic proof. The two independent fork tests supply that separate coverage.

Fragmented delivery executed under GasModel bounds. Stale-cursor bounds and standalone Safe grouping were verified by pure SDK planner tests; no deliberately stale delivery was executed through a Safe in this review. Stale delivery transactions remain separate Safe groups in the planner rather than being combined into an atomic batch whose later state assumptions may fail. Neither result proves the maximum practical wallet or miner registry for an unspecified launch.

7.3 Size and capacity boundaries

The production Pool runtime is 23,170 bytes, leaving **1,406 bytes** under the 24,576-byte limit. The distributor runtime is 8,137 bytes and its creation code is 9,597 bytes. Pool creation code is 35,120 bytes. Limits were not disabled to obtain a passing deployment. The separate recoverable canary leaves only **135 bytes** of runtime headroom, which warrants checking on any future canary change but is not a production defect by itself.

Admission still settles the existing registry before adding a miner. Registry-size gas growth and non-REVOKED settlement failures remain per-pool liveness constraints. Splitting miners across two pools changes the distribution of load; it does not bound either registry or create a migration mechanism. The fixed whitelist and permanent custody make a routing mistake consequential. Actual miner inventories, status transitions, largest allocation wallet and transaction/Safe configuration remain launch-specific checks.

8. Inherited risks, compiler and canary separation

8.1 Prior findings and SALUS remediation applicability

Table 9. SALUS remediation applicability

Original SALUS item	Current focused assessment
#1 - rewards locked after unsupported transfer/revocation before registration	Production code limitation remains. Owner accepted the fixed initial-holder, atomic whitelisted `depositMiners` policy. Actual holders, routing and absence of unsupported direct transfers are not verified for launch.
#2 - carried-income allocation depends on admission ordering	Per-miner carried-income remediation remains in unchanged accounting source. Fresh differential regressions and new Regular-only carry proof support it. Equal realized inputs are required for order claims; upstream timing, caps and rounding can change inputs.
#3 - centralization/finality	Genuine code gates first custody on relevant finalized/ownerless components. Fresh finality tests and both startup directions support continued operation. Setup authenticity, full allocation and irreversible early renunciation remain explicit boundaries.
#4 - unbounded wallet revocation	Inherited and owner-accepted under constraints. No arbitrary-wallet capacity guarantee. A shared mistaken allocation now affects both reward streams.
#5 - limited top-ID retraction after ownership/holding changes	Inherited and owner-accepted. Correct issuer-held inventory before irreversible movement/finality; new contracts do not recover old assets.
#6 - unsolicited receipt transfers delay finalization	Inherited and owner-accepted. Unexpected accepted inventory must be allocated before finalization, which can delay activation of both pools.
#7 / #8 - canary events and fixed emergency-return IDs	Remain separate canary-only observations. No fresh SALUS closure or production recovery power is inferred.

The 18 fresh Fable finding tests are local mock-protocol regressions. They confirm additional unchanged limitations: retained revoked-miner composition weights, non-REVOKED settlement blocking admissions, pre-composition BEM recognition under the first composition, transfer-hook income timing, a bounded carried-income representation, the one-raw-unit precision threshold, empty distributor finalization and early irreversible renunciation. The injected settlement-failure and transfer-hook cases demonstrate Pool behavior, not that genuine upstream Mining/NFT currently reaches those paths. A PASS means the documented behavior was reproduced. Their adapted fixture often operates only Optimal; they are not counted as symmetric two-pool proofs.

The underlying raw `claimableBatch` preview can overcount duplicate IDs; SDK canonicalization remains the operator protection, and actual claims do not pay an ID twice. Strict one-raw-unit interpretations of ideal rational entitlement remain subject to the existing scaled-integer threshold behavior. No accounting change in 2.3 repairs or worsens these arithmetic rules; the two pools apply them separately.

“Optimal” and “Regular” are labels, not an on-chain miner-quality predicate. Both implementations validate the approved collection, ACTIVE status, burn and supported categories. A permitted miner routed to the wrong pool stays in that pool's permanent custody. There is no production cross-pool move, withdrawal or rescue function. This newly relevant routing decision requires an explicit approved miner-to-pool map before admission.

8.2 Shared external dependencies

The pinned fork used BEM `0x5ce033B2bFCa3Af30b3e8C8457DeaF776A8b695a`, Mining `0x7E2E0DC66a3bD9103E69b766afA62d9f7b697b46` and miner collection `0xb1024b89886B9a34Aa4ff5F31C411D708b20a14C`. The upstream implementation/beacon identities were reviewed in the supplied historical material and fixture, but this task is not a complete audit of those external systems.

The fork proves compatibility with the selected historical state and tested miners. It does not establish latest public-chain authority, current code, future reward availability or the state at a prospective deployment. Earlier Fable and Astra observations about beacon ownership refer to different blocks and must not be collapsed into a current authority claim. This review did not perform a new latest-state authority inventory.

BEM issuance caps, actual claim returns, revocation behavior and common dependency changes can affect realized inputs and liveness. Two local ledgers do not make those external contracts independent. Recheck actual dependency addresses, code/implementation/beacon identities and reachable authority at a recorded launch block.

8.3 EX23-01 - Known compiler exposure remains unresolved

The official SOL-2026-7 advisory covers optimized solc **0.8.19 through 0.8.37** and is fixed in **0.8.38**; upstream rates it **Low**. This candidate uses **0.8.24, via IR, optimizer 200**, so it is within the affected settings class. Paris does not remove that exposure. Whether this exact build is miscompiled remains **UNVERIFIED**, not confirmed. [S15-S16]

The upstream explanation says source patterns alone cannot reliably exclude the issue. Successful tests and identical rebuilt artifacts do not close it. Before deployment consideration, use a reviewed fixed-compiler candidate or establish a defensible exact-build exclusion using the compiler fix. No fixed-version rebuild, exact-version backport comparison or exhaustive exclusion was executed

here. Changing the compiler/build belongs to a new separately reviewed candidate; the frozen implementation was preserved.

8.4 Canary-only assessment

`RecoverableTapeoutMinerPool/canary-4-local` propagates the production membership/version changes and adds a second canary pool for the two-member distributor. Its recovery patch list remains the historical canary patch set. The second pool is intentionally miner-less, and the helper is bound to the first pool. This does not prove both-pool contract-holder reward operation or production recoverability.

Source/hash and fresh artifact comparison succeeded. The generator `--check` did **not** complete: attempts encountered missing Forge, then missing Node under a restricted path, then a formatting child that was interrupted without a result. No canary generator PASS is asserted. A new complete/interrupted canary rehearsal was not executed. Developer records of 48 complete and 10 interrupted assertions remain author-recorded.

No new confirmed canary defect was found in the focused adaptation review. Existing canary-only events/fixed-ID issues, narrow runtime-size margin and orchestration dependencies remain separate. The historical mainnet approval/fingerprint did not authorize this candidate, and no live canary preparation, deployment, recovery or return transaction was attempted.

9. Fresh execution ledger and limits

9.1 Toolchain and preservation

Fresh execution used Foundry/Anvil/Cast **1.4.3**, commit `fa9f934bdac4bcf57e694e852a61997dda90668a`, native solc **0.8.24+commit.e11b9ed9**, **Paris**, **via IR**, optimizer **200**, Node **24.22.1** and npm **11.9.0**. The locked packages include OpenZeppelin 5.4.0, viem 2.57.2, zod 3.24.2, tsx 4.19.3 and TypeScript 5.7.3. These are this review's tools; the developer's Darwin/Node 20 run is a separate record.

The final check found **all 788 tracked files** in the pristine frozen checkout identical to their starting SHA-256 values, with a clean Git worktree and the exact target HEAD. **All 158 original execution inputs** under source, SDK, CLI, scripts, tests, libraries and the selected configuration/lockfiles matched the pristine target. Review proofs and generated outputs are separate. Evidence: `evidence/final-preservation.json`, `evidence/toolchain-final.json` and `logs/source-manifest-start.json`.

9.2 Completed tests

125 distinct Solidity tests passed: 122 local/offline tests and 3 real-protocol fork tests. Four of the local tests executed **1,100 differential fuzz cases** in total, with zero failed corpus replays. This count uses actual named results, not intended command filters, duplicated reruns or whole developer pipeline totals.

Table 10. Freshly executed checks

Fresh group	Result	Evidence / scope
New independent local two-pool proofs	13/13	Asymmetric arithmetic, staggered startup, transfer, callbacks, rollback, selective failures, carry, small units, topology and mimic setup.
Existing TwoPoolDistributor	17/17	Constructor, membership, both finality states, history rejection, complete delivery and claim/transfer behavior.
GasModel	11/11	Isolated-call bounds, zero-history issue/funding and fragmented delivery.
PoolSetup + SalusFinality	27/27	Nine setup and eighteen finality tests.
Distributor + Receipt + PoolClaims	31/31	Fifteen distributor, five receipt and eleven claim tests.
Fable findings + differential	20/20	Eighteen limitation/regression tests plus 100 small-amount and 300 general model cases.
AuditDifferential + SalusDifferential	3/3	400 audit-model cases, 300 SALUS-model cases and the exact SALUS example.
Existing TwoPoolFork + two independent fork tests	3/3	Same pinned real-protocol block; the two stronger tests cover both delayed-start directions and transfer.
Existing SDK TypeScript	79/79	Actual named TAP cases; zero failures/skips.
Audit TypeScript	10/10	Actual named TAP cases.
Audit2 TypeScript	3/3	Named rerun without Node test isolation resolved an earlier file-level-only result.
New independent SDK/CLI probes	20/20	Actual frozen functions with in-memory clients/offline CLI. Some PASS results confirm findings.
Full local CLI E2E	79 checks / 90 steps	Unmodified workflow, local mock protocol.
Safe fork E2E	294 checks / 28 Safe transactions	Both-pool setup; paid reward flow only Optimal. 47 total local fork transactions.
Build, artifact regeneration, typecheck	PASS	Fresh 43-file build; all six ABI/bytecode exports match; TypeScript typecheck succeeds.

The three FFI targets were inspected: `test/audit/model.py`, `test/salus-ffi/model.py` and `test/review-fable51/model_fable.py`. They import numeric/system standard-library components, process numeric argv traces and write ABI-encoded outputs; they do not access files, invoke subprocesses or use the network. Their invoking Solidity uses fixed `python3 -I` argument arrays. These are single-pool ledger reference models within the adapted fixture, not independent evidence of the entire two-pool lifecycle.

9.3 Failed attempts, repairs and reruns

Table 11. Failed attempts and retained qualifications

Attempt	What happened	Treatment
Public source/dependency acquisition	Restricted DNS/proxy environment and an npm configuration collision prevented initial acquisition; a later npm attempt lacked the platform CA.	Correct public proxy/CA handling and separate empty npm configs resolved installation. TLS validation was retained. No candidate test failure.
New proof naming	A review-only name beginning `testFailed...` hit Foundry's removed `testFail*` convention before execution.	Renamed only that new test to `testFirstAdmissionFailure...`; assertions unchanged; 13 tests then passed. Initial record retained.
Test-selection helper	Substring skip `Distributor.t.sol` also excluded `TwoPoolDistributor.t.sol`.	Detected from actual suite output, fixed review helper, ran all 17 existing tests separately. The initial run was never counted as 30 tests.
Default loopback restrictions	Initial local CLI/socket-dependent runs failed or were interrupted before meaningful completion.	Authorized isolated-loopback execution succeeded; first attempts remain separate environmental records.
Independent tooling harness	Initial review directory lacked an ESM marker.	Fixed review-only harness metadata; no candidate file changed; 20 probes completed.
First real-protocol run	Existing fork test passed; two new tests failed fetching archive storage after upstream HTTP 429 responses.	Throttled read-source service and reran the two new tests at the same block. Both passed; initial two infrastructure failures retained.
Audit2 TAP visibility	First command returned one file-level PASS, without the three named cases.	Reran the unchanged file with `--test-isolation=none`; all three named cases appeared and passed. Counted only the named result.
Canary generator	Missing executable paths, then an interrupted formatting child with no completed result.	NOT EXECUTED to completion. No generator PASS or new canary rehearsal claimed.

No failed candidate assertion was removed or suppressed to obtain the final results. The fork retry addressed provider rate limiting without altering code, state fixtures, block or assertions. Service start/stop interruptions are operational records, not smart-contract failures. Full commands, durations, exit codes and raw outputs are retained in the evidence archive.

9.4 Author-recorded results and unexecuted checks

The developer package records 244 main Solidity tests, 10 audit FFI, 2 SALUS FFI and 20 Fable review tests; its GasModel rerun duplicates 11 tests already counted in the main group. It also records 79 SDK and 13 audit TypeScript cases, 79 local CLI checks, 21 fork tests, 26 fork CLI checks, 294 Safe checks and the canary scenarios. The package correctly labels the overall run **INCOMPLETE** because Slither was skipped. All of these historical counts remain AUTHOR-RECORDED unless a distinct fresh result is listed above.

An independent implementation of its code-fingerprint algorithm hashed 170 relevant tracked files to `5beb9b98851030d337374ba0c4723f2a6da4e345868b4dd2fbb66b6a9b2940a0`, matching the target. This is meaningful input correspondence despite the original run being based on a dirty bdf906f working tree. It does not prove execution or replace the missing raw logs.

The following checks remain explicitly incomplete or outside this focused execution: the full 244-test main pipeline and all invariant campaigns as a single run; all 21 historical fork tests and the separate 26-check fork CLI workflow; a new canary complete/interrupted recovery rehearsal; static analysis with Slither, which is absent as executable/module; actual launch-sized miner/wallet capacity; actual beneficiary contract usability; original contribution truth and allocation approval; current external authority/code inventory; and a definitive compiler-bug exclusion. The review completed the supported source analysis and targeted execution without asking for credentials or substituting public-chain experiments.

10. Coverage matrix

“Both” below means the stated evidence covers the pair or the relevant role explicitly; it does not imply every inherited test was duplicated for both addresses. The raw named-test ledger is authoritative for exact test scope.

Table 12. Both-role critical transition coverage

Critical transition or property	Optimal	Regular	Evidence and limitation
Genuine deployment, distinct roles, common receipts/protocol	Executed	Executed	Local CLI, constructor unit tests, real fork.
Zero/code-less/duplicate/incompatible pool rejection	Pair-wide	Pair-wide	Existing two-pool constructor tests; categories also covered by setup tests.
Fund before either pool activates	Executed	Executed	Local proofs, fork and Safe zero-history paths.
One full allocation and no pool beneficiary	Pair-wide	Pair-wide	Existing 17-test suite and local CLI; inventory is not duplicated.
One-time authorized binding and third-member rejection	Executed	Executed	Two-pool and finality tests; direct setup can choose another distributor.
Correct finalized supply and allocation before activation	Executed	Executed	Two-pool tests; retained setup/finality suite often uses Optimal as primary.
Shared owner gates, own pool renunciation	Executed	Executed	Two-pool unit tests and both fork startup directions.
Other pool unbound/inactive/owner-held	Executed first	Executed first	Two independent fork tests; Regular-first local proof adds mock assertions.
Delayed bind after shared renunciation	Executed delayed	Executed delayed	Both fork directions.
Failed first admission atomic rollback	Primary local proof	Peer independence checked	Detailed mock rollback proof targets the primary pool; same immutable implementation in both. Not two separate symmetric fault tests.
Actual distinct real miner admission and income	Executed	Executed	Fork miners 12733 and 13948; pinned state only.
Late delivery and independent claims	Executed	Executed	Literal local oracle and enhanced forks.

Critical transition or property	Optimal	Regular	Evidence and limitation
Reverse claim order and unequal paid history	Executed	Executed	Local proof and second-ID fork sequence.
Receipt transfer and new-holder additional income	Executed	Executed	Local exact 50/140 BEM example and genuine-protocol fork.
NAND/LATCH separation and operator denial	Executed	Executed	Local/fork and retained claim tests.
Consumed-history funding rejection	Executed	Executed	Real paid histories tested against a second unbound genuine distributor.
Callback claims / later rejection rollback	Executed	Executed	One receiver claims both; later callback failure rolls back both nested effects.
Selective harvest/admission failure	Healthy peer	Injected failure	Mock only; failed harvest item is reported, next admission reverts.
Selective sink failure	Healthy peer	Injected A+C failure	Mock only; no claim that genuine BEM has sender-selective behavior.
Zero income, tiny rounding, zero LATCH shares	Executed	Executed	Independent local tests; exact fixture bounds.
Actual CLI selection and independent reconciliation	Executed	Executed	Local mock-protocol E2E plus targeted SDK probes.
Safe setup and shared batches	Executed	Executed	Fresh 294 checks; Safe reward lifecycle only Optimal.
Actual launch identities/beneficiaries/capacity	Unverified	Unverified	Missing launch inputs; examples and historical canary are insufficient.

11. Prioritized disposition and remaining launch inputs

11.1 Confirmed blockers and gates

No confirmed new intrinsic two-pool inability-to-run blocker was found. Further local testing can use this preserved candidate. Production deployment consideration should remain on hold for the following concrete assurance gates, which predate the shared-distributor change or concern actual setup rather than a newly demonstrated ledger exploit:

1. **Resolve T23-01 and T23-02 in the release verification path.** Require complete five-role and instantiated-runtime/constructor assurance before funding, binding and admission as applicable, with phase-appropriate validation for earlier issuance/finalization. Finalizing issuer-held receipts need not wait for future pool/distributor addresses. A separately reviewed verifier or equivalent independent exact-deployment evidence can supply the missing assurance; an unqualified current `MATCHES` cannot. If code is changed, use a new frozen target and regress both phases and roles.

2. **Resolve EX23-01 before deployment of optimized affected output.** Produce a reviewed fixed-compiler build or a defensible exact-build exclusion. This is an inherited compiler gate with upstream Low severity, not a demonstrated candidate exploit.
3. **Establish the selected launch topology and full shared allocation before funding/finalization/admission as appropriate.** The tested split-binding/empty-round counterexample means flags alone are insufficient. If a real setup is already wrong after a one-time step, assess that actual state; no migration or recovery should be assumed.

The missing raw author logs and absent Slither are evidence limitations to close where practical, but are not themselves confirmed reasons the supported two-pool route cannot run. The uncompleted canary generator check does not invalidate the freshly built production artifacts or create production return powers.

11.2 Required launch evidence, not new feature requests

Before funding, independently approve original contribution/allocation data, complete category/ID coverage, both receipt supplies, every beneficiary and usable recipient control. Before allocation finalization, reconcile accepted inventory and per-ID mappings and resolve unwanted inventory or incorrect ranges while the remaining correction mechanisms are still available.

Before binding/activation, authenticate the five intended addresses, exact runtime/constructor inputs, ordered pool roles, shared receipt/protocol references and intended distributor. Before each first admission, record that pool's activation/supplies/binding and owner-zero state plus finalized/ownerless shared contracts. Verify pending owners are cleared, legitimate deliveries and transfers are accounted for, and no unexplained NFT/BEM custody exists.

Approve an explicit miner-to-pool routing list, the fixed depositor configuration, actual initial holdings/approvals and admission feasibility at the intended registry sizes. Verify both A/B recipient sets independently; a status note is insufficient. Rehearse the actual largest wallet and Safe batches with the actual receiver types. Record current external code/authority at a block and the selected compiler/build provenance. These inputs were not supplied as a production launch package and remain UNVERIFIED.

The owner-confirmed constraints for SALUS #1/#4/#5/#6 remain relevant. Their new shared impact should be acknowledged in the actual release decision, especially that one bad receipt allocation can affect both streams. This report does not request a product redesign, treat those historical acceptances as code fixes, or infer a new deployment authorization from them.

12. Evidence navigation, reproduction and source references

12.1 Evidence package

The companion archive contains review-only local/fork proof sources, the independent SDK/CLI probe, inspected execution wrappers, named test results, raw logs including failed attempts, fresh fork/Safe/local-CLI observations, artifact and immutable-group comparisons, source-preservation manifests, consolidated execution metadata and a SHA-256 file inventory. The machine-readable review summary records findings, evidence classes, statuses and explicit unexecuted checks. Installed dependencies, compiler binaries, caches, ephemeral runtime directories and production credential files are not bundled.

Table 13. Companion evidence navigation

Evidence	Location in companion archive
Independent 13 local contract proofs	`contracts/IndependentTwoPoolContractReview.t.sol`
Independent two real-protocol scenarios	`evidence/IndependentTwoPoolFork.t.sol`
Twenty actual SDK/offline CLI probes	`tooling/tooling_review.ts`, `tooling/tooling-review-results.json`
Every fresh named test and exact command	`evidence/fresh-execution-manifest.json`, `logs/*-result.json`
Inspected FFI regression results and fuzz counts	`evidence/offline-regression-summary.json`
Actual fork block/hash and observations	`logs/fork-throttled-preflight.json`, `logs/fresh-review-two-pool-*.json`
Safe checks, transactions and gas	`tooling/fresh-fork-safe-e2e.json`, `tooling/safe-execution-manifest.json`, `tooling/safe-setup-gas-results.json`
Fresh CLI workflow and reconciliation	`logs/fresh-local-e2e.json`, `logs/fresh-local-e2e-reconcile.json`
Source/build equality and final preservation	`evidence/provenance-source-bytecode.json`, `evidence/fresh-artifact-verification.json`, `evidence/final-preservation.json`
Environment and failed attempts	`evidence/toolchain-final.json`, `logs/`, `tooling/initial-run-failure.json`, `evidence/canary-check-attempts.json`

Reproduction requires a fresh isolated archive of the exact target and the pinned tools. The included `run_forge_selected.py` selects test files, forces offline solc 0.8.24 compilation with one job and seed `0x02ef1f7`, and preserves full results through `run_logged.py`. Copy the two independent Solidity files into `test/independent/` in that archive. Do not copy them into a frozen production checkout. Use the exact commands recorded in each result JSON, replacing only local paths to tools/evidence as necessary.

The four regression groups select PoolSetup/SalusFinality, Distributor/EntitlementReceipt/PoolClaims, Fable findings/differential with `--ffi`, and AuditDifferential/SalusDifferential with `--ffi`. The SDK suite uses Node's test runner with tsx and

TAP; the three Audit2 cases require the recorded `--test-isolation=none` invocation to reproduce the visible named output in this environment. The local CLI and Safe scripts were run unmodified after inspection. Fork reproduction must verify a loopback endpoint and the stated block hash and retain upstream code/storage. Never substitute a public write endpoint merely because it reports chain ID 56.

12.2 Frozen source and primary external references

All repository links below use `02ef1f7da17683b482b97b28464346bc2cb9db61`. Line references in the report refer to this frozen revision, not current main. Historical reports retain their historical scope.

- **S1:** TapeoutMinerPool.sol. Constructor/authentication 250-286; binding/activation 298-328; claims 418-440 and 817-839; role movement 460-468; history views 521-552; admission/carry/finality 625-790.
- **S2:** TapeoutReceiptDistributor.sol. Constructor 91-110; hooks 115-143; allocation/revocation 153-198; finality/delivery 203-235; history check 297-307; cursor/effect/transfer ordering 314-336.
- **S3:** PoolMath.sol and SDK arithmetic.
- **S4:** TapeoutEntitlementReceipt.sol. Issue/retract/finalize 58-99; holder-index update 131-155.
- **S5:** Deployment verification and configuration schemas.
- **S6:** CLI, allocation verification and transaction endpoint guards.
- **S7:** Pool SDK, gas model and deployment script.
- **S8:** Artifact exporter, generated bytecode and IReceiptDistributor.
- **S9:** Accounting, architecture and operations.
- **S10:** Prior Fable review and actual prior Astra PDF.
- **S11:** Audit assumptions and owner acceptance.
- **S12:** SALUS remediation report and original SALUS PDF.
- **S13:** Two-pool change report and developer evidence directory.
- **S14:** Two-pool unit tests, original two-pool fork test, GasModel and verification runner.
- **S15:** Official Solidity SOL-2026-7 advisory, retrieved during this review.
- **S16:** Official Solidity known bugs list, SOL-2026-7 entry, retrieved during this review.
- **S17:** BNB Chain BEP-652, official transaction gas-cap specification, retrieved during this review. The candidate's budget is additionally confirmed from its frozen gas source; future network conditions require rechecking.

13. Direct answers to the owner's six questions

13.1 Can this exact candidate initialize one shared distributor and operate both pools after the required renunciations?

Yes, within the reviewed genuine-code and setup assumptions. Fresh local CLI execution, 13 independent local proofs, 17 existing two-pool tests and three real-protocol fork tests establish executable routes. Funding can read inactive pools' zero histories, each pool can later bind/activate/renounce, and both operate with shared owners at zero. No circular setup dependency was found. This does not validate an unspecified deployment or remove the identity/compiler gates.

13.2 Can every legitimately allocated receipt be delivered once and earn/claim independently from both pools, including after transfer?

Yes for a correct finalized allocation to a usable receiver, within the tested limits and available income. Delivery is once-only, late income remains attached, paid histories stay separate, and unpaid/new income follows the current holder. The exact 125/350 BEM example, unequal-history transfer and later 50/140 BEM example passed. This is not an unconditional promise for an incapable beneficiary, incorrect allocation, arbitrary unbounded wallet, failed common token or unverified external conditions.

13.3 Does staggered startup or failure in one pool unnecessarily block the other?

No intrinsic peer-startup dependency was found. Both directions passed with the delayed peer initially unbound, inactive and owner-held, and delayed binding after shared renunciation. Local selective-failure proofs preserved the functioning peer and finalized delivery. Shared setup errors, bad identity, deliberately coupled receiver callbacks and common upstream failures can still affect both; they are distinct from cross-pool ledger coupling.

13.4 Are on-chain bindings and operational identity checks sufficient under the stated setup assumptions? What remains trusted?

They support the intended route only with authenticated implementations, a complete operational manifest and independently verified launch inputs. On-chain membership is not reciprocal topology enforcement or code authentication. Finalized allocation is not full issued-supply proof. The full-manifest checker catches tested role/binding mistakes, but T23-01/T23-02 prevent treating generic `MATCHES` as complete identity assurance. Constructor provenance, all immutable occurrences, both current recipient sets, correct original JSON, receiver usability and miner routing remain trusted until independently established.

13.5 Are there confirmed High/Medium defects, asset-locking paths or practical inability-to-run issues introduced or worsened by the change?

No new confirmed High/Medium honest-route production defect or intrinsic inability-to-run issue was found. Two inherited Medium verification weaknesses remain, with increased two-pool scope. Irreversible setup-locking paths remain real: premature renunciation with unfinished allocation, finalized wrong beneficiaries or unusable receivers, and irreversible wrong-pool routing. A still-owned incomplete allocation may be completable. Shared receipt mistakes now affect rights from both pools. The genuine split-binding/empty-round counterexample confirms a setup boundary; it is not an outsider attack after correct binding. Known compiler exposure remains an inherited unresolved gate, not a demonstrated asset exploit.

13.6 What must be fixed before further deployment consideration, and what is inherited or unverified?

Close the operational completeness/runtime-authentication gaps and the compiler gate, then establish the actual five-contract launch topology, full shared allocation and per-pool inputs before their irreversible steps. Use a new frozen candidate for any code/build changes and retest both roles. Historical SALUS #1/#4/#5/#6 constraints, permanent custody, registry-capacity limits, rounding and common-dependency risks remain inherited limitations with the documented qualifications. Actual launch recipients, contribution data, receiver capabilities, miner routing, gas capacity and current dependency authority remain unverified inputs. No earlier PASS, owner-controlled mainnet canary or this qualified review grants production deployment authorization.